

Talos RAG

Owner's Manual

How the retrieval system is configured, built, used, operated, and backed by evaluation.

The whole system in one sentence. One pipeline, driven by one config object (`RagConfig`), exposed through one endpoint (`/ask`) that streams to the asker and broadcasts to the channel, and measured by an eval harness that calls the **same** pipeline code. Files and chat conversation live as vectors in one Milvus collection (`talos_documents`), told apart by a `source` field — chat memory is embedded as **conversation segments**, not single messages. Master three chokepoints — **RagConfig** (all knobs), **build_rag_pipeline** (all retrieval), **RagTrace** (all observability) — and you can scale, debug, and fix any part of it.

Branch `feature/chat-message-memory` · worktree `~/talos-main` · `src/rag/` + `src/processing/`

1 · System overview

Talos RAG has exactly one flow: **writers** put vectors into a single Milvus collection; **readers** query them to answer a question. There is no second pipeline hiding anywhere — even the evaluation harness reads through the same core (bottom of the diagram). One thing flows **out** as well: when /ask finishes an answer, it broadcasts it to the channel's Socket.IO room as an `ai_message` event, so every member sees it, not just the asker.

1 · System overview — one collection, writers on the left, readers on the right

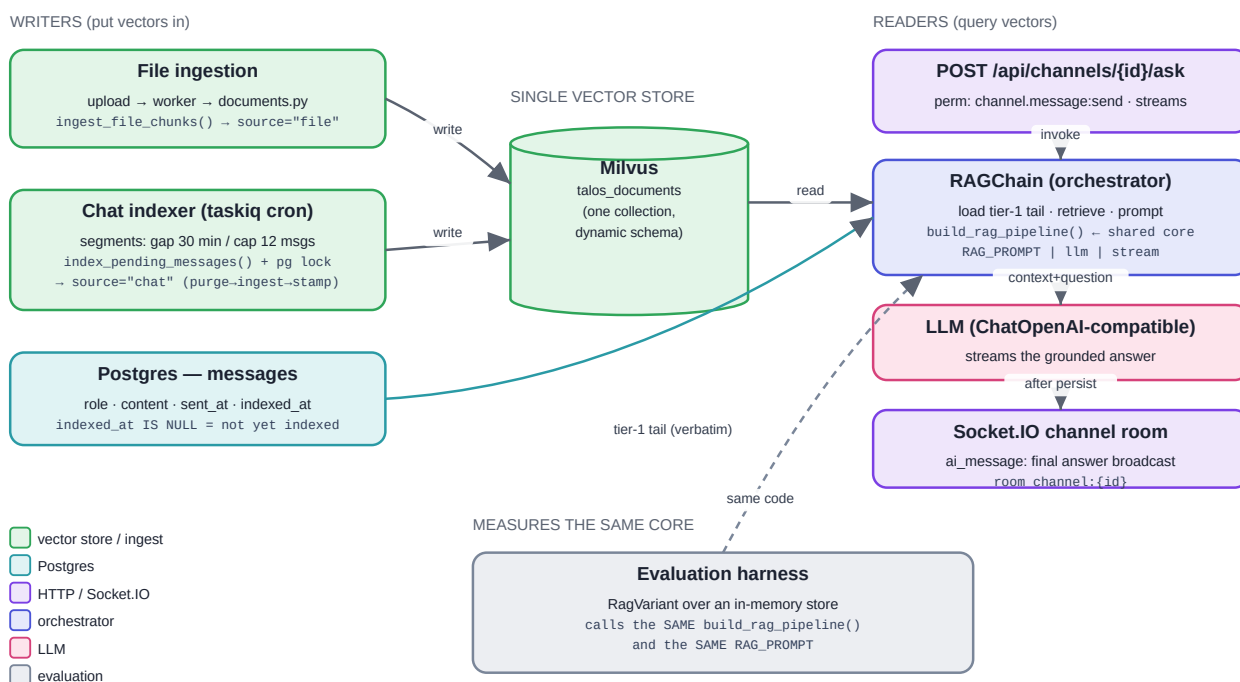


Diagram 1: Writers on the left, one vector store in the middle, readers on the right. Evaluation shares the core; finished answers fan out to the channel room.

Key idea. Everything funnels through `talos_documents`. File chunks are tagged `source="file"`; chat-memory segments are tagged `source="chat"`. Retrieval filters on that field, so the two streams never bleed into each other.

There is only one RAG entry point in the app — `POST /api/channels/{id}/ask`. Ordinary chat (`POST /api/channels/{id}/messages`) just stores and broadcasts a message; it never calls the LLM. So /ask is additive and self-contained.

Measured, not claimed. Live-verified on the dev stack (2026-07-02): while an /ask generates, an unrelated endpoint answers in **~1 ms** — including the very first request after boot, when the embedding model and reranker load (~10 s of work, all off the event loop). Before this architecture the same scenario froze every request for 6–10 seconds.

2 · Configuration — the single source of truth

Every knob is a field on `RagConfig` in `src/config/config.py`, instantiated once as `global_rag_config`. It reads from environment variables (and `.env`); the field name upcased is the variable name. There is

no YAML for RAG config. To change behaviour you set an env var, or change the default here — that is the only place.

Field (env var)	What it does	Default
openai_model	Answer + query-rewrite LLM	gpt-4o-mini
embedding_provider / embedding_model	How text becomes vectors	openai / text-embedding-3-small
milvus_collection_name	The one collection (app + CLI agree)	talos_documents
use_query_rewrite	Rewrite the query before retrieval (1 LLM call)	True
use_hyde	Hypothetical-document embedding for the query (1 LLM call)	True
use_reranking	Cross-encoder reranks candidates	True
rerank_fetch_k	Candidate pool fetched before reranking down to top-k	20
use_hybrid_retrieval	Add BM25 lexical channel (needs a corpus)	False
compression_type / ..._similarity_threshold	Post-retrieval context compression	none / 0.76
retrieval_top_k	Final file chunks fed to the prompt	5
chat_context_cap	Tier-1 tail: max un-indexed messages injected verbatim	50
chat_recall_k / chat_recall_fetch_k	Tier-2: final segments kept / candidate pool fetched	3 / 10
chat_decay_half_life_hours	Recency half-life for recall re-ranking (0.25 floor)	168 (1 week)
chat_recall_overlap_threshold	Jaccard overlap above which a candidate is redundant	0.6
chat_segment_gap_minutes / chat_segment_max_messages	Segment boundary: inactivity gap / size cap	30 / 12
chat_index_interval_minutes / _grace_seconds	Indexer cadence / settle window	5 / 300
chat_index_batch_size / chat_index_max_batches	Messages per batch / batches drained per tick	500 / 10

Key idea. The config isn't just read at startup — it is **injected** through every factory (config=). That is what lets one process run several configs at once, which is exactly how evaluation works (Diagram 5A).

3 · Component map

Everything lives under src/rag/ (plus the indexer in src/processing/). Each file has one job.

File	Responsibility
config/config.py	RagConfig — all knobs; global_rag_config
rag/vector_store.py	Milvus connection, get_embeddings (cached), get_workspace_vectorstore, deletes (incl. delete_chat_segments_for_messages), WORKSPACE_COLLECTION
rag/ingestion.py	ingest_file_chunks (source=file), ingest_chat_messages (source=chat), format_citations

File	Responsibility
rag/message_text.py	The ONE seam turning Message.content into text (already handles rich-msg's ProseMirror dicts)
rag/retrieval/retrievers.py	build_rag_pipeline — the shared file-retrieval composition
rag/retrieval/query_processing.py	get_query_rewriter, get_hyde_embeddings
rag/retrieval/compression.py	compression_retriever
rag/retrieval/chat_selection.py	select_chat_context — pure decay + redundancy re-ranking of recalled segments
rag/generation.py	get_llm
rag/rag_chain.py	RAGChain — orchestrator; prepare() (eager retrieval) + stream_answer() (generation); fills the trace
rag/trace.py	RagTrace — the observability record
rag/router.py	POST /ask: tail loading, threading, persistence, ai_message broadcast
processing/chat_indexing.py	build_chat_segments + index_pending_messages — the locked cron indexer
processing/chat_tasks.py	The taskiq task: cron schedule, retry label, multi-batch drain

4 · Lifecycle of one /ask request

RAGChain is split in two, and the split is the async-correctness story. **prepare(question)** does everything retrieval: rewrite, HyDE, file search, chat-segment recall, context formatting — synchronously, but the router runs it (together with chain construction) in a worker thread via `asyncio.to_thread`. If anything in retrieval fails, prepare raises **before any response bytes are sent**, so the client gets a real 502, not a broken stream. **stream_answer(prepared)** only generates: it streams LLM tokens, then fills the trace. The router iterates it with `iterate_in_threadpool`, so no token ever blocks the event loop.

2 · Lifecycle of one /ask request

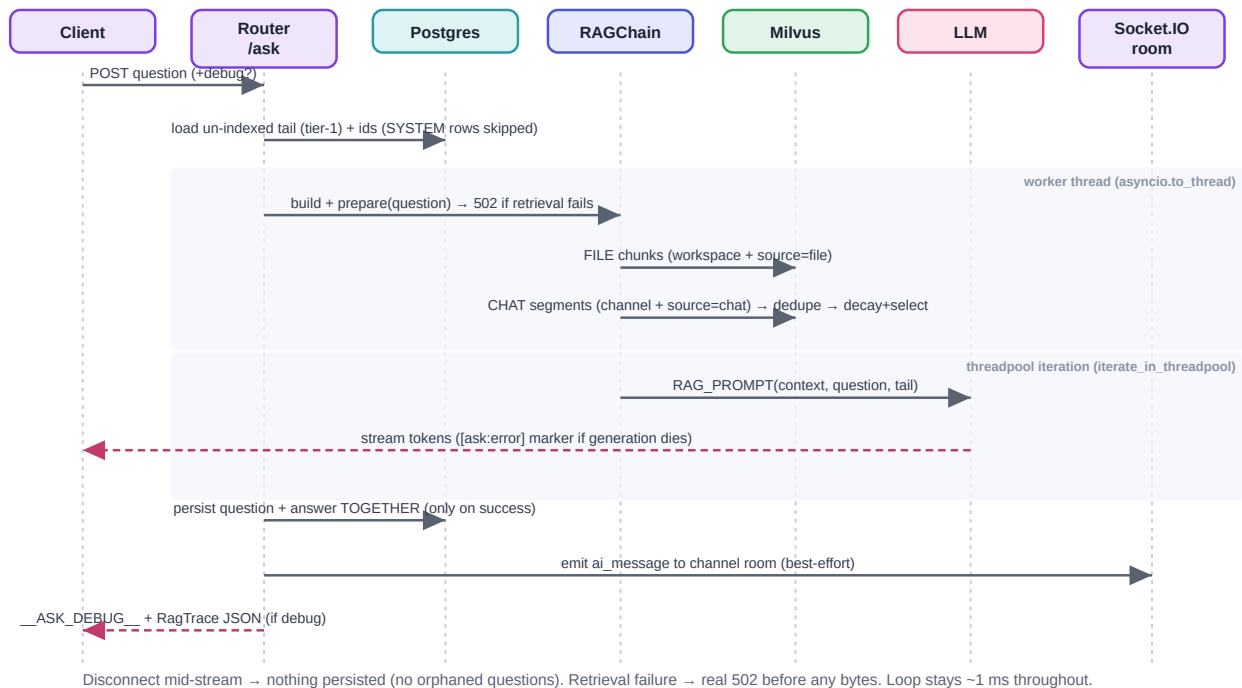


Diagram 2: One request, top to bottom. Shaded bands run off the event loop. Dashed red = streaming back to the caller.

After a **successful** stream, the router persists the user question and the assistant answer **together, in one commit** — and only then emits `ai_message` to the channel room. The consequences are deliberate:

- **Client disconnects mid-stream** → nothing is persisted. No orphaned questions ever pollute the channel's history or future prompts.
- **LLM dies mid-stream** → the (already-200) stream ends with the marker `[ask:error]`, and nothing is persisted. Clients can detect the marker.
- **Milvus/rewriter down** → clean 502 before headers.
- **Broadcast fails** → logged warning; the request still succeeds (best-effort by design).

The `ai_message` payload is a custom event (the chat message schema requires a human sender): `{channel_id, question_message_id, message_id, question, content, role: "assistant"}` on room `channel:{id}`.

5 · Two-tier chat memory

This is the one non-obvious idea, so hold it clearly. A channel's conversation is split at the **indexer boundary**:

- **Tier 1** — recent messages the indexer hasn't touched yet (`indexed_at` IS NULL, SYSTEM notices excluded). Injected into the prompt **verbatim**, capped at `chat_context_cap`.

- **Tier 2** — older conversation the indexer has embedded into Milvus as **segments**. Recalled semantically, then re-ranked (§5.2).

A message is in exactly one tier. The router passes the tail's ids as `exclude_message_ids`; a recalled segment is dropped if **any** of its `message_ids` overlaps the tail, so a message momentarily on both sides (its vector lands a beat before its `indexed_at` commit) is still counted once.

3 · Two-tier chat memory — every message is in exactly one tier

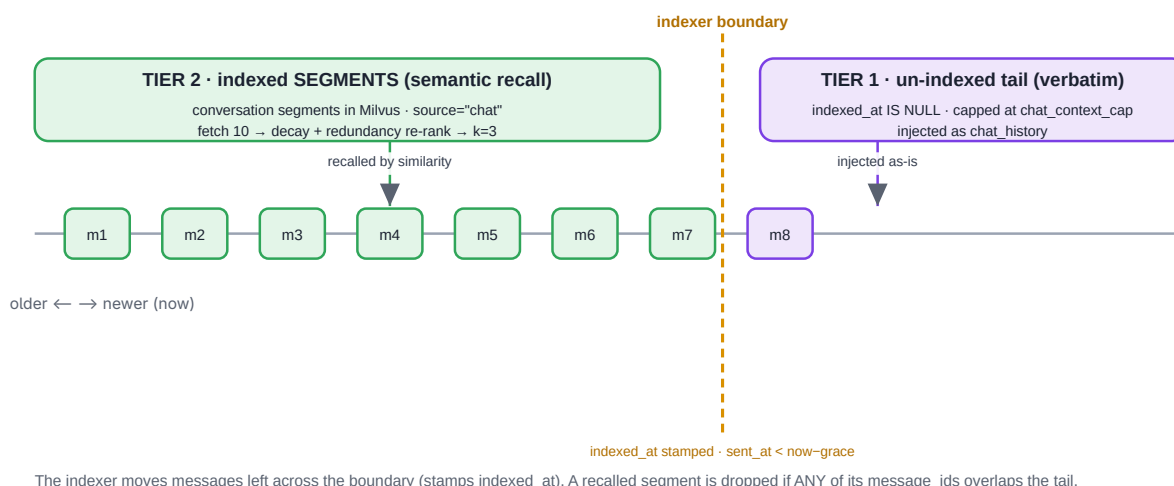


Diagram 3: The indexer stamps messages and moves them from tier 1 (verbatim) to tier 2 (semantic recall as segments).

5.1 Why segments, not messages

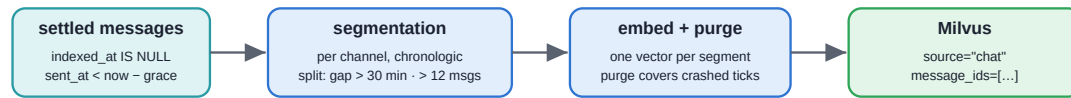
A lone “yes, let’s do that” embeds meaninglessly — per-message vectors fragment conversations below the level where similarity search can work. The indexer therefore groups settled messages into **per-channel, chronologic segments**, closed by an inactivity gap (`chat_segment_gap_minutes`) or a size cap (`chat_segment_max_messages`). The evidence: SeCom (ICLR 2025) showed topic-coherent multi-turn segments beat both turn-level and session-level retrieval units under the same retriever (GPT4Score 71.57 vs 65.58 vs 63.16 on LOCOMO). An inactivity gap is the cheapest online-safe boundary proxy: zero extra LLM or embedding calls. Each segment’s metadata carries the full `message_ids` list — that is what powers both the tail dedupe and idempotent purging.

5.2 Query-time selection: decay + redundancy

Recall fetches a **wide** candidate pool (`chat_recall_fetch_k`), then a pure function (`select_chat_context`) re-ranks it: $\text{score} = \text{rank_relevance} \times (0.25 + 0.75 \cdot 0.5^{(\text{age}_h / \text{half_life})})$ — so recency matters but an old, uniquely relevant segment never decays to zero — and a greedy pass skips candidates whose token Jaccard overlap with an already-picked segment exceeds the threshold (near-duplicates waste context). The survivors (`chat_recall_k`) join the prompt. The whole recall path is wrapped in one guard: **any** failure degrades to file-only context and a warning log. Chat memory can never kill an answer.

4 · Chat memory — segments at index time, smart selection at query time

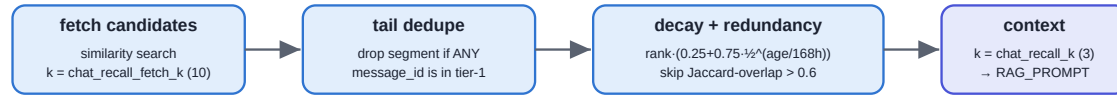
INDEX TIME (taskiq cron, every chat_index_interval_minutes, advisory-locked)



indexed_at is stamped only after ingest succeeds — a crashed tick re-selects the same batch and the purge removes its partial segments (idempotent).

Why segments: a lone "yes, let's do that" embeds meaninglessly; topic-coherent multi-turn segments outperform per-message units (SeCom, ICLR 2025).

QUERY TIME (inside RAGChain.prepare, per /ask)



Selection is a pure function (select_chat_context) — and the whole recall path degrades to file-only context on ANY failure; it can never kill the answer.

Diagram 4: Index time: settled messages → segments → vectors. Query time: fetch wide → dedupe vs tail → decay + redundancy → k segments.

6 • build_rag_pipeline() — the shared file-retrieval core

All file-retrieval logic lives in one function. Read it once and you understand every retrieval path — production and evaluation both call it.

5 • build_rag_pipeline() — the one shared retrieval composition

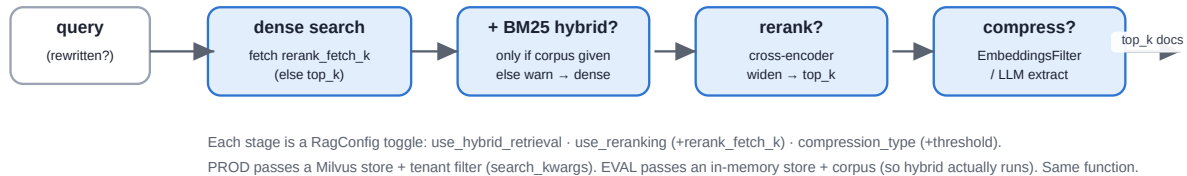


Diagram 5: Dense → optional hybrid → optional rerank (with widening) → optional compression. Each stage is a config toggle.

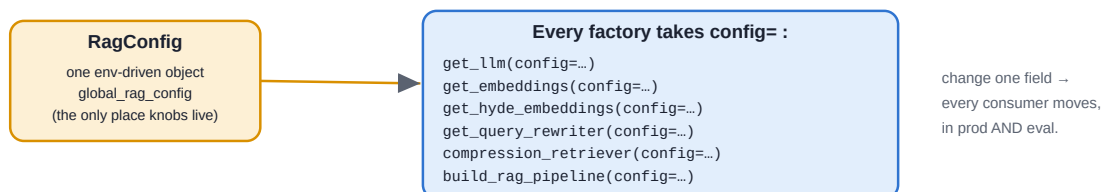
Key idea. Reranking is only useful because the dense stage fetches rerank_fetch_k (wide) and the cross-encoder narrows to retrieval_top_k. Hybrid needs a corpus (BM25 is lexical) — production has none, so it warns and falls back to dense; evaluation passes the corpus, so hybrid genuinely runs there.

7 • The three chokepoints

The whole system is deliberately funnelled through three things you can hold in your head at once. Learn these and nothing is a mystery.

6 • The three chokepoints you must hold in your head

A • RagConfig → reaches every component through a real config= seam



B • RagTrace → filled once per run → read by every debug surface (one schema)



C • build_rag_pipeline → the only place retrieval logic lives (see diagram 4). Edit it once; production and evaluation both change.

Master these three — all knobs, all retrieval, all observability — and you can scale, debug, and fix any part of the system.

Diagram 6: A: one config reaches everything. B: one trace is read by every debug surface. C: one retrieval function.

- **A — RagConfig** is the only place knobs live, and it reaches every component through a real config= seam.

- **B** — **RagTrace** is filled once per run and read identically by the `/ask debug` flag, `scripts/debug_ask.py`, and the chat UI. One schema, no drift.
- **C** — **build_rag_pipeline** is the only place file-retrieval logic lives. Edit it once; production and evaluation both change.

8 · Operating the indexer

The chat indexer is a taskiq cron task (`processing/chat_tasks.py`) executed by the worker; the scheduler only enqueues ticks. Its correctness story:

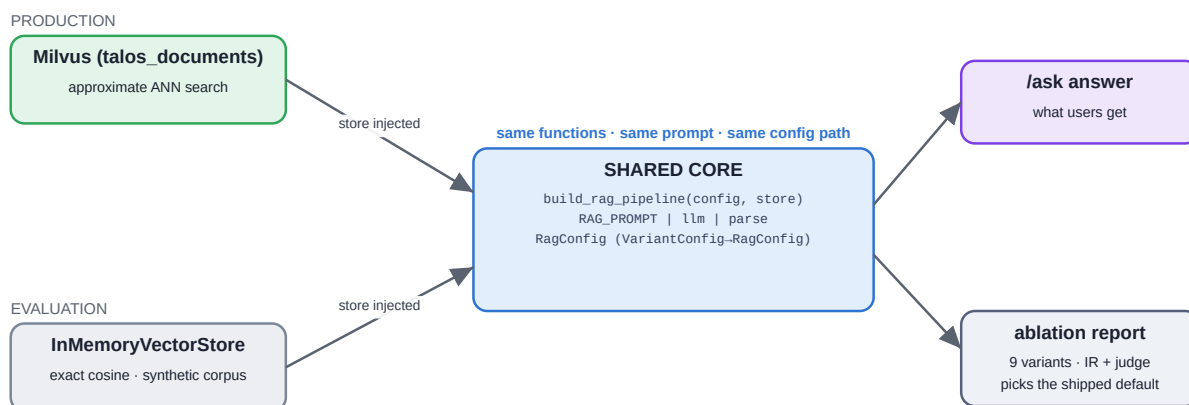
- **Concurrency.** `index_pending_messages` takes Postgres session-level advisory lock `0x7A105C47` on a **dedicated connection** held for the whole run (the ORM session's pooled connection can change across commits — the lock must not live there). A second concurrent run logs and returns 0. If the process crashes, Postgres releases the lock with the connection.
- **Idempotency.** Order is `purge → ingest → stamp`. A crashed tick leaves rows un-stamped; the next tick re-selects the **same oldest batch** (deterministic `ORDER BY sent_at LIMIT batch`) and the purge (`json_contains_any` over the batch's `message_ids`) removes any partial segments first. Segments are batch-local, so no segment ever spans two batches.
- **Throughput.** Each tick drains up to `chat_index_max_batches × chat_index_batch_size` messages (default 5,000/tick), stopping early on a short batch — a backlog burst clears in one tick.
- **Retries.** The task carries `retry_on_error=True, max_retries=3` — three immediate attempts (the Redis stream broker ignores delay labels), then the next cron tick is the durable fallback. Nothing is stamped on failure, so retries are safe.

Key idea. Ops invariants: run exactly **one** scheduler instance (taskiq requirement — two schedulers double every tick), and remember index lag is **masked**, not fatal: un-indexed messages ride verbatim in tier 1. Migration from legacy per-message vectors (optional): delete `source == "chat"` vectors, run `UPDATE messages SET indexed_at = NULL;`, let the indexer re-embed segments. Legacy vectors also keep working for recall and dedupe in the meantime.

9 · Evaluation — “what we evaluate is what we ship”

The eval harness (`tests/rag_evaluation/`) builds an in-memory index over a synthetic Q&A set and runs an ablation grid of 9 variants. Crucially, each variant retrieves via the **production** `build_rag_pipeline` and generates with the **production** `RAG_PROMPT`; its flags are translated into a real `RagConfig`. The `production_default` row is **derived from** `global_rag_config`, so the headline number always reflects the deployed configuration.

7 · Eval == ship — evaluation drives the production code



`production_default` is DERIVED from `global_rag_config`, so the measured row is always the deployed config.
The one deliberate difference is the store: approximate ANN (prod) vs exact cosine (eval).

Diagram 7: Production and evaluation meet at the shared core. The only difference is the vector store.

The workflow to pick defaults is data-driven: run the grid → the winning variant's flags **become** the shipped `config.py` defaults. Metrics include IR (Hit/Recall/Precision/MRR/nDCG@k), LLM-judge (faithfulness, relevancy, correctness), bootstrap CIs, and paired significance tests.

Honest gaps you should know as the owner: (1) hybrid retrieval is measured in eval but cannot run in production (no BM25 corpus is passed) — flipping `use_hybrid_retrieval` on from eval numbers silently ships dense-only; (2) the chat-memory tiers (tail injection, segment recall, selection) have no quantitative eval coverage yet — they are covered by unit tests and live verification, not the ablation grid. Both are known, documented, and the next evaluation milestone.

10 · Debugging playbook

Every query fills `chain.trace` (a `RagTrace`): model, effective config, rewritten query, file/chat candidates, injected-tail size, final context, and the **exact prompt**. Read it three ways: send `{"debug": true}` to `/ask` (it appends `__ASK_DEBUG__` + JSON to the stream), run `scripts/debug_ask.py <channel> "<question>"`, or use the chat UI's 🔍 toggle.

Symptom	Check the trace / logs → likely cause
502 from <code>/ask</code>	Retrieval failed before streaming: Milvus down, rewriter LLM down, bad collection. Log line <code>ask retrieval failed</code> .
Stream ends with <code>[ask:error]</code>	Generation died mid-stream (LLM). Nothing was persisted — the turn simply didn't happen. Log line <code>ask generation failed</code> .
Empty / “can't answer”	<code>file_candidates empty</code> → nothing ingested, wrong <code>workspace_id</code> , or legacy rows missing the source key (re-ingest).

Symptom	Check the trace / logs → likely cause
Chat not remembered	chat_candidates empty → messages still in tier 1 (not indexed yet), indexer lagging (warn log: tail hit cap), or recall degraded (warn log chat recall failed with chatroom_id).
ai_message never arrives	The socket only joins rooms for channels visible via role permissions on connect — check the user's roles and reconnect. The emit itself is best-effort (warn log on failure).
Indexer “does nothing”	Log chat indexer lock held elsewhere → another run is active (or a leaked lock: check pg_locks for key 0x7A105C47).
Milvus dimension error	Embedding provider/model changed against a populated collection → drop + re-ingest.
Hybrid “does nothing” in prod	Expected — no BM25 corpus in prod; use evaluation to measure hybrid.
Slow answers	HyDE + rewrite = two extra LLM calls per query → turn off via USE_HYDE / USE_QUERY_REWRITE.

Known limits (own them before anyone asks)

- The tier-1 cap counts **messages**, not tokens — one pasted wall of text is uncapped. A token budget is the natural next hardening step.
- Under heavy backlog (>50 un-indexed messages in a channel) messages 51+ are temporarily in **neither** tier until the indexer catches up (minutes, bounded by the drain rate; warned in logs).
- Segments never span indexer batches — a conversation straddling a 500-message batch cut becomes two segments (rare; quality nit, not correctness).
- The question's sent_at uses the app clock; the answer's uses the DB clock. Ordering relies on generation time exceeding clock skew (same-host: fine).

11 · Recipes — how to change it

- **Flip a feature:** set the env var (USE_HYDE=false, USE_RERANKING=false, USE_HYBRID_RETRIEVAL=true) or change the default in config.py.
- **Swap model / embeddings:** OPENAI_MODEL, EMBEDDING_PROVIDER / EMBEDDING_MODEL. New provider → extend _build_embeddings in vector_store.py.
- **Tune the memory:** segment shape via CHAT_SEGMENT_GAP_MINUTES / CHAT_SEGMENT_MAX_MESSAGES (re-index to apply to old data); recall behaviour via CHAT_RECALL_FETCH_K, CHAT_DECAY_HALF_LIFE_HOURS, CHAT_RECALL_OVERLAP_THRESHOLD; freshness via CHAT_INDEX_INTERVAL_MINUTES / _GRACE_SECONDS.
- **Add a retrieval stage** (MMR, a second filter, ...): edit build_rag_pipeline once; production and evaluation both get it, and eval will measure it.
- **Tune rerank recall:** raise RERANK_FETCH_K (wider pool) vs RETRIEVAL_TOP_K (final count).
- **Change the prompt:** src/config/prompts.py (RAG_PROMPT) — eval uses the same object.
- **Survive the rich-msg migration:** Message.content becomes a ProseMirror document — reading is already funnelled through rag/message_text.py (one file to review); the one write site (_persist_exchange in rag/router.py) must switch to set_content() when that branch lands.
- **Re-pick defaults with data:** run the eval grid, set config.py to the winner.

Three chokepoints: **RagConfig** · **build_rag_pipeline** · **RagTrace**.
One boundary: the indexer. One guarantee: chat memory can never kill an answer.